

RoadGen3D: From Design Intent to Reviewable 3D Street Scenes

Rule/Constraint-Driven, AI-Assisted Street Scene Generation and Evaluation

Shiqi Wang GIStudio

HKUST(GZ) Society Hub
Urban Governance and Design (UGOD)

GIStudio Internal Project · 2026

Urban Design

3D Generation

GeoAI

Presenter: Shiqi Wang · Faculty Project Presentation · 2026-07-10



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

社会枢纽
SOCIETY HUB

RoadGen3D studies a reviewable generation pipeline, not merely a 3D viewer

One-sentence position

A **rule- and constraint-driven, AI-assisted** system for generating and evaluating 3D urban street scenes.

- How does an input become specific roads and facilities?
- Which rules pass, and where does a design fail?
- Why do alternatives differ, and can they be reproduced?

Core contribution path

Design intent → **Explicit IR** → **3D artifacts**
→ **Evaluation**

Evidence: docs/ROADGEN3D_FRAMEWORK.md; docs/current-progress.md

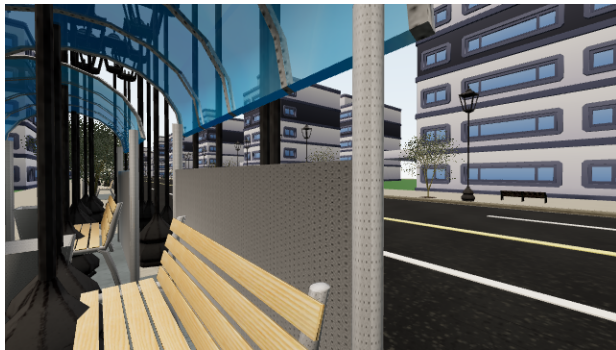
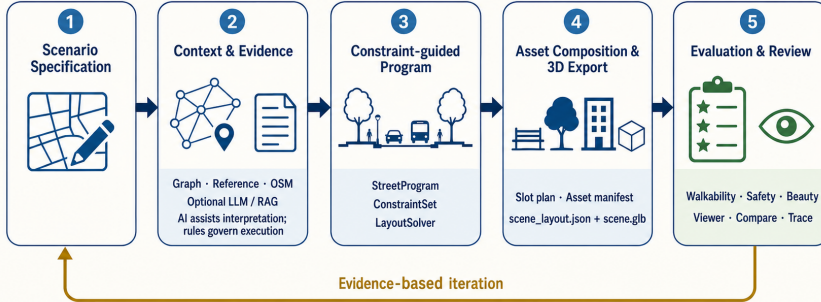


Figure 1: A semantic edit becomes a bus shelter, seating, road geometry, and building frontage. The image is one form of result evidence.

The system compiles design intent into evaluable scenes and retains human review

RoadGen3D: From Design Intent to Reviewable 3D Street Scene



AI assists interpretation; rules govern execution. Evaluation informs the next scenario iteration rather than creating an unsupervised decision loop.

Explicit intermediate representations separate natural language from 3D geometry

Intermediate representation	Question answered
<code>DesignDraft</code>	What does the user require, and which fields come from explicit input, a preset, or evidence?
<code>SceneContext</code>	Does the run use a graph template, reference, OSM input, or a compatibility template?
<code>StreetProgram</code>	How are cross-sections, functional bands, demands, and furniture goals represented?
<code>ConstraintSet</code>	Which requirements are hard rules and which are soft objectives?
<code>LayoutSolverResult</code>	What are the band widths, slot plans, conflicts, and fallbacks?
<code>scene_layout.json</code>	Which facts can the Viewer, evaluation, diff, and reproduction workflows share?

Evidence: docs/DATA_CONTRACTS.md; src/roadgen3d/services/design_types.py; src/roadgen3d/types.py

Why not prompt-to-mesh?

Street design must preserve **clear sidewalk width, transit edges, furniture spacing, refuge islands, collision checks, and rule evidence**. A final mesh alone cannot explain these decisions.

Two outputs, distinct roles

- `scene.glb`: rendering and interchange
- `scene_layout.json`: factual and evaluation contract

AI interprets intent and retrieves evidence; deterministic rules govern execution

AI / RAG support

- Intent parsing and retrieval queries
- PDF / GraphRAG / parameter triples
- Parameter suggestions and field-level citations
- Optional visual evaluation and recommendations

Rules and contracts

- Compose-patch allowlist
- Template-patch validation
- Hard and soft constraints
- Geometry, collision, and carriageway-intrusion vetoes

Traceable outputs

- Final values and source priority
- Citations and overridden candidates
- Requested and used backend
- Fallback reasons and warnings

Key invariant

Manual and explicit inputs outrank LLM suggestions. If a learned component or RAG is unavailable, record the fallback; never substitute silently.

Evidence: `src/roadgen3d/llm/design_workflow.py`; `src/roadgen3d/services/design_runtime.py`; `generation_trace.json`

The road skeleton bounds the space; the solver compiles demands into slot plans

1. Graph / Reference Bridge

base annotation + `template_patch`

→ `RoadSegmentGraph` + `PlacementContext`

2. Program + Constraints

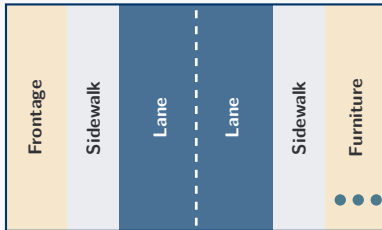
`StreetProgram` describes bands, demands, and objectives. `ConstraintSet` compiles the design rules.

3. Layout Solver

Produces band widths, slot plans, rule evaluations, conflicts, and fallback provenance.

Evidence: `src/roadgen3d/template_patch.py`; `graph_template_scene_bridge.py`; `design_rules.py`; `layout_solver.py`

Cross-section bands + allowed placement zones



Example rules

Preserve minimum clear sidewalk width; place bus stops only on transit edges; keep seats outside the stopping envelope; prevent assets from entering the carriageway.

Assets are not scattered randomly: demands, retrieval, filtering, and placement jointly determine the result

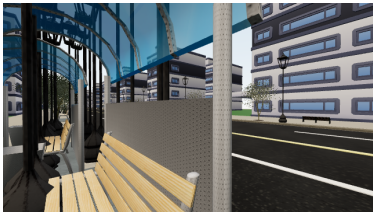


Figure 2: Human scale: bus shelter and seating



Figure 3: Overall composition: stops, lights, and buildings

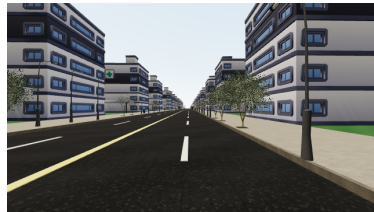


Figure 4: Continuous street frontage

Furniture requirements → Slot plan → Manifest retrieval → Geometry / rule filters → Placement + mesh export

Failure is also data

Unplaced assets, collisions, carriageway intrusions, side mismatches, and constraint vetoes remain as diagnostics rather than being forced into the scene.

Evidence: `src/roadgen3d/street_layout.py`; `placement_decisions.jsonl`; `capture_manifest.json`

The plan, GLB, and trace jointly form reproducible scene evidence



Figure 5: Scene Map Plan: roads, surfaces, buildings, and furniture are driven by the same manifest.

Recommended run directory

- `scene_layout.json`
- `scene.glb`
- `generation_trace.json`
- `production_steps/`
- `placement_decisions.jsonl`
- `view_captures/`

Data authority

A GLB can be rebuilt or replaced.
Only **layout** + **provenance** explains
where the scene came from.

Evidence: `artifacts/full_flow_smoke/.../20260709T153713Z; web/viewer/src/viewer-api.ts`

Evaluation separates structural evidence from visual evidence and degrades honestly when inputs are missing

Structural evidence

`scene_layout.json`

- Cross-sections and clear widths
- Slot, placement, and overlap records
- Rule satisfaction, conflicts, and feasibility
- Walkability and generation quality

Visual evidence (optional)

Rendered views / capture manifest

- Safety and beauty judgments
- Multiple views, child view, and overview
- LLM status and visual-input status
- Scenario-specific rubrics and semantic gates

Evaluation integrity

Without valid rendered views or an evaluator, Safety, Beauty, and Overall remain **N/A**. Missing evidence is never disguised as a zero or a complete aggregate score.

Evidence: docs/EVALUATION.md; viewer-evaluation-runner.ts; eval_engine_ext/road_metrics

A faculty-facing live demo should cover intent, mechanism, result, and evaluation within four minutes

- 0:00 Position:** The Viewer is a review surface. Begin with the final scene and production steps.
- 0:35 Edit:** Enter “Add a 24 m green bus platform on the right side of the arterial’s middle segment.”
- 1:15 Compile:** Show the semantic edit, resolved defaults, and validated template patch.
- 1:55 Run:** Submit the job; show monotonic progress, the stage tree, and parameter decisions.
- 2:40 Review:** Load a pre-generated layout; switch among Plan, 3D, and production steps.
- 3:25 Evaluate:** Show metrics, N/A degradation, comparison, and trace.

Preflight

```
make api
make viewer-web
GET /api/health
```

Contingencies

- LLM unavailable: `use_llm:false`
- Slow generation: switch to a pre-generated artifact
- Missing visual score: discuss structural evidence only
- Job state lost on restart: load the layout from disk

Evidence: Makefile; docs/DEPLOYMENT_AND_JOBS.md; semantic_scenario_edits.py

Collaborators integrate through explicit contracts rather than bypassing the IR to edit meshes

Role	Recommended point	entry	Primary input / output	Boundary to pre-serve
Planning / design	Scenario catalog, semantic edit, template patch		scenario intent, functional zones, surface annotations	Do not write meshes directly; validate every patch
API / application	/api/design/draft, /api/scene/jobs		JSON request, job polling, result / trace	Keep Draft separate from SceneContext
Generation methods	Program, Constraint, Solver, asset backend		IR, slot plan, candidate / placement logs	Record requested / used backend and fall-back
Evaluation / analysis	scene_layout.json, rendered views		metrics, rubric, comparison, improvement patch	Never fabricate scores without visual evidence
Frontend / external tools	Viewer ?layout=, Road-Pen bridge		ViewerManifest, GLB, bridge summary / losses	Vite file API is local only; bridge loss must be auditable

Evidence: docs/ACTIVE_ENTRYPPOINTS.md; tools/bridge/README.md; web/api/schemas.py

Minimal API contract: submit a structured job, poll its status, and read the artifacts

```
# 1. create
curl -X POST http://127.0.0.1:8010/api/scene/jobs \
  -H 'Content-Type: application/json' \
  --data-binary @scene_job.json
# -> {"job_id":"...", "status":"queued"}

# 2. poll
curl http://127.0.0.1:8010/api/scene/jobs/$JOB_ID
# -> status, stage, progress, operations,
# result, evaluation, trace
```

Create request

- draft
- scene_context
- patch_overrides
- generation_options

Success result

- layout / GLB / PLY paths
- summary + compose config
- viewer URL
- generation trace + evaluation

Evidence: POST /api/scene/jobs; GET /api/scene/jobs/{job_id}; src/roadgen3d/services/scene_jobs.py

The system demonstrates a controllable generation pipeline, but not yet complete road-engineering capability

Implemented Demonstrable now

- Scenario catalog, semantic edit, and graph templates
- Explicit IR, hard / soft rules, and slot solver
- Asset retrieval, collision filtering, GLB + layout
- Viewer Plan / 3D / compare / history
- Structural evaluation, optional visual evaluation, and trace

Conditional Conditional or experimental

- LLM, PDF RAG, and GraphRAG require configuration and artifacts
- Learned program / placement requires a checkpoint and runtime
- Safety / Beauty requires valid visual input
- OSM, MetaUrban, and reference paths still need contract convergence

Next Claims not yet supported

Not covered: complete lane movements, signal control, traffic simulation, or production-grade multi-user jobs. Explicit structure alone does not establish a validated neuro-symbolic model.

Validate effectiveness before expanding model and road-engineering complexity

1 Freeze the benchmark and golden artifacts

Make scenarios, seeds, asset manifests, rendered views, evaluation profiles, and expected results fully reproducible.

2 Strengthen road-engineering evaluation

Add lane connectivity, turn movements, crossing exposure, reachability, delay, and simulation-based safety.

3 Reassess the need for learned components

Raise learned or neuro-symbolic claims only after adding training data, checkpoints, baselines, ablations, and a closed loop with the Viewer.

Questions for faculty feedback

- Which should the first benchmark prioritize: **rule compliance**, **design differentiation**, or **evaluation stability**?
- Which interface has the greatest research value: scenario specification, the constraint solver, or the evaluation protocol?

RoadGen3D

From design intent to **controllable,**
traceable, and evaluable 3D street scenes Q & A

Appendix A: Key data contracts and factual outputs

Contract	Core content	Primary consumer
DesignDraft	normalized query, config patch, citations, risk, parameter sources	design runtime
SceneContext	layout mode, graph/reference/OSM, scenario, template patch	context bridge
StreetProgram	cross-section, demands, furniture requirements, objectives	constraint compiler
ConstraintSet	declarative hard/soft rules, profile, weights	layout solver
LayoutSolverResult	band solution, slot plans, conflicts, fallback provenance	placement / trace
scene_layout.json	config, IR, placements, semantic layers, outputs, summary, scene graph	Viewer / evaluation / diff

Appendix B: Most important current interfaces

Generation

- POST /api/design/draft
- POST /api/scene/jobs
- GET /api/scene/jobs/{id}
- POST /api/scenario-designs/runs
- POST
/api/scenario-designs/draft-variant

Review / Integration

- POST /api/design/evaluate/unified
- POST /api/design/evaluate/compare
- POST /api/design/capture-views
- Viewer ?layout=<scene_layout.json>
- tools/bridge/format_bridge.py

Deployment boundary

The most reliable current topology uses HTTP plus an artifact contract on one machine or a shared file system. A remote artifact registry, persistent job table, and worker pool are not yet productionized.

Appendix C: Internal invariants worth challenging

- ① **Contract separation**: Compose patches, SceneContext, and generation options remain separate.
- ② **Template immutability**: Patch a deep copy of the base graph; a variant never changes the source template.
- ③ **IR before geometry**: Text and LLM outputs never place meshes directly.
- ④ **Explainable fallback**: Record the requested backend, used backend, and reason together.
- ⑤ **Placement safety**: Enforce carriageway intrusion, collision, target strip, side, and rule veto checks.
- ⑥ **Visual-score honesty**: Do not return a falsely complete score without visual input.
- ⑦ **Artifact duality**: GLB is the visual carrier; layout is the factual contract.

Appendix D: Bridge boundaries between RoadPen and RoadGen3D

Reliably interoperable

- road topology / centerlines
- node coordinates / scale
- approximate widths / lane counts
- geometry-preserving roundtrip checks

May degrade, but must be recorded explicitly

- regions / building regions
- surface annotations
- station strips / junction composition
- furniture relationships
- warnings, losses, repaired, __bridge_meta

Principle

The bridge prioritizes **geometry preservation and auditable semantic degradation**; it does not claim lossless transfer of all semantics.